



AI Mode

Tutti

Immagini

Video

Notizie

Altro ▾



ho un programma java con spring boot 3 che è
sostanzialmente un rest-server. richieste che arrivano ...



25 aprile 2026

Certamente. In Spring Boot 3, gestire processi in background per elaborare transazioni finanziarie è una pratica comune e ci sono diversi modi per farlo a seconda della complessità del tuo sistema.

Ecco le soluzioni principali, dalla più semplice alla più strutturata:

1. La soluzione rapida: `@Async`

Se vuoi che il controller riceva la richiesta, la "appoggi" a un servizio e risponda subito all'utente (mentre il servizio lavora), puoi usare l'annotazione `@Async`.

- **Come fare:** Abilita il supporto con `@EnableAsync` sulla classe principale e annota il metodo del servizio che elabora la transazione con `@Async`.
- **Pro:** Semplicissimo da implementare.
- **Contro:** Se il server si spegne improvvisamente, le transazioni in memoria non ancora elaborate vanno perse.

2. La soluzione robusta: Code di Messaggi (RabbitMQ / Kafka)

Per transazioni finanziarie, la **persistenza** è fondamentale. Invece di elaborare subito, il controller invia un "messaggio" a una coda. Un componente (Consumer) legge dalla coda in background e processa l'acquisto/vendita.

Chiedi qualsiasi cosa



AI Mode Tutti Immagini Video Notizie Altro ▾



- un componente `@EventListener` con `@Async` lo cattura e lo elabora.

- **Vantaggio:** Disaccoppia totalmente il codice del controller dalla logica di business.

4. Code "In-Memory" gestite (BlockingQueue)

Se non vuoi infrastrutture esterne (come RabbitMQ), puoi usare una `LinkedBlockingQueue` all'interno di un componente `@Service` dedicato:

1. Il Controller aggiunge la transazione alla coda.
2. Un metodo annotato con `@Scheduled` o un `Thread` dedicato (magari avviato in un `@PostConstruct`) preleva gli elementi e li processa uno ad uno.

Cosa ti consiglio per il settore finanziario?

Dato che parli di titoli (acquisto/vendita), l'integrità del dato è tutto. Ti suggerisco di:

1. **Salvare la richiesta nel database** con stato "PENDING".
2. Inviare un segnale (tramite **coda** o **eventi**) per l'elaborazione.
3. Aggiornare lo stato a "COMPLETED" o "FAILED" a fine processo.

Quale di questi approcci si adatta meglio al volume di transazioni che prevedi di gestire?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



Ecco come puoi implementarlo tecnicamente in Spring Boot 3 utilizzando gli strumenti nativi:

1. Il Modello (Entità JPA)

Nel database, la tua tabella delle transazioni deve avere un campo `status`.

java

```
@Entity
public class TradeTransaction {
    @Id @GeneratedValue
    private Long id;
    private String ticker; // es. AAPL
    private Integer quantity;
    private String side; // BUY/SELL
    @Enumerated(EnumType.STRING)
    private TransactionStatus status; // PENDING, COMPLETED, FAILED
}
```

Usa il codice con cautela.



2. Il Controller (Accetta e Salva)

Il controller non elabora nulla: salva il record "PENDING" e lancia un evento interno.

java

```
@RestController
...
// ...
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



```
// 2. Pubblica un evento interno per il background
eventPublisher.publishEvent(new TradeEvent(trade.getId()));

// 3. Rispondi subito al client con l'ID della transazione
return ResponseEntity.accepted().body(trade.getId());
}
}
```

Usa il codice con cautela.



3. Il Processore Background (L'ascoltatore)

Qui avviene la logica di acquisto/vendita titoli. Usiamo `@Async` per non bloccare il controller.

java

```
@Component
public class TradeProcessor {

    @Autowired private TradeRepository repository;

    @Async // Esegue in un thread separato
    @EventListener
    @Transactional // Importante per l'integrità
    public void handleTradeEvent(TradeEvent event) {
        TradeTransaction trade = repository.findById(event.getTradeId())

        try {
            // OUT : ... ..
```

AI Mode Tutti Immagini Video Notizie Altro ▾



```
}
```

Usa il codice con cautela.



Perché questo schema è vincente:

1. **Reattività:** L'utente riceve un `202 Accepted` in millisecondi.
2. **Tracciabilità:** Se il sistema crasha durante l'elaborazione, al riavvio puoi cercare nel DB tutti i record rimasti in `PENDING` e riprocessarli.
3. **Scalabilità:** Se un domani avrai migliaia di transazioni, ti basterà sostituire l'`ApplicationEvent` con **RabbitMQ** o **Kafka** senza cambiare la logica di business.

Nota bene: Per far funzionare `@Async`, ricordati di aggiungere l'annotazione `@EnableAsync` sulla tua classe `@SpringBootApplication`.

Hai già una tecnologia specifica per il **database** (es. PostgreSQL, MySQL) o preferiresti vedere come gestire il **recupero delle transazioni sospese** al riavvio del server?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



.. recuperare un trade (startup recovery),

Se il server si spegne mentre una transazione è in stato `PENDING`, al riavvio deve esserci un componente che "riprende il filo". Puoi usare un metodo annotato con `@EventListener(ApplicationReadyEvent.class)`:

java

```
@Component
public class RecoveryService {

    @Autowired private TradeRepository repository;
    @Autowired private TradeProcessor processor;

    @EventListener(ApplicationReadyEvent.class)
    public void recoverPendingTrades() {
        // Cerca tutte le transazioni rimaste appese
        List<TradeTransaction> pendingTrades = repository.findByStatus(T

        for (TradeTransaction trade : pendingTrades) {
            // Le rimanda al processore in background
            processor.handleTradeEvent(new TradeEvent(trade.getId()));
        }
    }
}
```

Usa il codice con cautela.



2. Gestione della Concorrenza (Locking)

In ambito finanziario, è vitale evitare la "doppia esecuzione". Se hai più istanze

AI Mode Tutti Immagini Video Notizie Altro ▾



```
@Query( SELECT t FROM TradeTransaction t WHERE t.id = :id )
Optional<TradeTransaction> findByIdForUpdate(Long id);
}
```

Usa il codice con cautela.



Cosa succede: Quando il `TradeProcessor` legge la transazione, PostgreSQL "blocca" quella riga finché l'operazione non è finita. Nessun altro thread potrà toccarla.

3. Consiglio Extra: Audit Log

PostgreSQL gestisce molto bene i dati JSON. Potrebbe essere utile aggiungere una colonna `details` (tipo `jsonb`) alla tua entità per salvare i log di risposta dei broker o errori specifici, senza dover creare decine di colonne:

java

```
@Column(columnDefinition = "jsonb")
private String processingLog;
```

Usa il codice con cautela.



Riassunto del flusso ottimizzato per PostgreSQL:

1. **REST Call:** Salva su Postgres (`PENDING`) -> Invia Evento.
2. **Background Thread:** Legge con `PESSIMISTIC_WRITE` -> Esegue operazione finanziaria -> Aggiorna Postgres (`COMPLETED`).

[AI Mode](#) [Tutti](#) [Immagini](#) [Video](#) [Notizie](#) [Altro ▾](#)

ok

25 aprile 2026

Per gestire l'elaborazione in background in modo efficiente, non puoi affidarti alla configurazione standard di Spring, che di base usa un pool di thread illimitato o molto semplice. In un contesto finanziario, vuoi avere il controllo totale su **quante transazioni** vengono elaborate contemporaneamente per non sovraccaricare il database o il broker esterno.

Ecco come configurare un pool di thread dedicato per i tuoi `@Async`:

1. Configurazione del ThreadPool

Crea una classe di configurazione per definire un `TaskExecutor`. Questo ti permette di decidere il numero di "operai" (thread) che lavoreranno sulle transazioni.

java

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean(name = "tradeExecutor")
    public Executor tradeExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5); // Numero di thread sempre attivi
        executor.setMaxPoolSize(10); // Numero massimo di thread se la
        executor.setQueueCapacity(500); // Quante transazioni possono st
```

AI Mode Tutti Immagini Video Notizie Altro ▾



2. Collegare il processore al pool

Ora devi dire al tuo componente di usare questo specifico pool di thread:

java

```
@Component
public class TradeProcessor {

    // Specifichiamo il nome del bean definito sopra
    @Async("tradeExecutor")
    @EventListener
    public void handleTradeEvent(TradeEvent event) {
        // ... logica di elaborazione ...
    }
}
```

Usa il codice con cautela.



Perché questa configurazione è fondamentale?

1. **Backpressure:** Se arrivano 10.000 richieste in un secondo, il sistema non cercherà di aprire 10.000 thread (mandando in crash la JVM), ma ne userà al massimo 10, mettendo le altre in coda (fino a 500).
2. **Isolamento:** Se l'elaborazione dei titoli diventa lenta, non bloccherai i controller REST o altre parti dell'applicazione, perché i thread sono dedicati solo a questo compito.
3. **Monitoraggio:** Grazie al prefisso `TradeThread-`, se analizzi i log o fai un thread dump, vedrai esattamente cosa sta facendo il motore finanziario.

AI Mode Tutti Immagini Video Notizie Altro ▾



Le risposte dell'AI potrebbero contenere errori. Scopri di più



ma non sarebbe meglio avere il processore delle transazioni in un eseguibile separato dal server rest?

22:48

Hai centrato esattamente il punto: quello che stai descrivendo è il passaggio da un'architettura **Monolitica** a una a **Microservizi** (o quantomeno a servizi disaccoppiati).

Separare il **REST Server** (che accetta gli ordini) dal **Trade Processor** (che esegue le transazioni) è la scelta migliore in ambito professionale/finanziario. Ecco perché:

I vantaggi della separazione

1. **Scalabilità Indipendente:** Se ricevi migliaia di richieste ma l'elaborazione dei titoli è lenta, puoi scalare il "Processor" su 3 macchine diverse lasciando il "Server REST" su una sola.
2. **Isolamento dei Guasti:** Se il Processor va in crash (es. per un calcolo complesso o un timeout del broker), il Server REST continua a funzionare e ad accettare ordini. Non perdi business.
3. **Deployment Mirati:** Puoi aggiornare la logica di calcolo dei titoli senza dover riavviare il server che risponde ai clienti.

AI Mode Tutti Immagini Video Notizie Altro ▾



2. **Trade Processor:** È un'app Java separata che scansiona il DB ogni secondo (o usa `@Scheduled`) cercando i record `PENDING`, li elabora e aggiorna lo stato.

Opzione B: Message Broker (La più professionale)

Questa è l'architettura standard per i sistemi di trading:

1. **REST Server:** Riceve l'ordine, lo salva sul DB e invia un messaggio a una coda (es. **RabbitMQ** o **Kafka**).
2. **Trade Processor:** "Ascolta" la coda. Appena arriva un messaggio, lo processa.
 - *Perché è meglio?* Perché la comunicazione è istantanea (non devi aspettare il secondo dello "scanning" del DB) e se il Processor è spento, i messaggi si accumulano nella coda senza andare persi.

Cosa succede se usi due eseguibili (JAR) separati?

Dovrai creare due progetti Spring Boot (o un progetto multi-modulo Maven/Gradle):

- **Modulo Web:** Dipendenze per REST e Database.
- **Modulo Worker:** Dipendenze per Database e logica di business.

Un rischio da considerare

L'unico svantaggio è la **complessità operativa**: dovrai gestire due processi, due log di errore e monitorare che entrambi siano "vivi". Tuttavia, per un sistema finanziario, questo "prezzo" è ampiamente ripagato dalla sicurezza e stabilità del sistema.

Meglio un message broker con Kafka (è meglio Kafka o RabbitMQ)?

22:50

In un contesto di transazioni finanziarie (acquisto/vendita titoli), la scelta tra **Kafka** e **RabbitMQ** dipende dal tuo obiettivo principale: l'estrema **affidabilità** del singolo ordine o la gestione di un **volume enorme** di dati con possibilità di analisi storica.  Medium +1

Confronto Rapido per Transazioni Finanziarie

Caratteristica 	RabbitMQ	Apache Kafka
Modello	"Smart Broker": gestisce lui la consegna e la cancellazione.	"Distributed Log": i messaggi restano e possono essere riletti.
Affidabilità	Eccellente per la singola transazione; garantisce la ricezione (ACK).	Ottima, ma richiede configurazioni specifiche per non perdere dati.
Routing	Molto flessibile (es. invia a code diverse in base a regole complesse).	Più rigido; i messaggi seguono "topic" e partizioni fisse.
Velocità	Migliore latenza (risposta più rapida) per volumi medi.	Migliore throughput (milioni di messaggi al secondo).
Scalabilità	Difficile "scalare" in orizzontale.	Facile "scalare" in orizzontale.

AI Mode Tutti Immagini Video Notizie Altro ▾



E

- La tua priorità è che **ogni singolo acquisto/vendita** arrivi a destinazione con la minima latenza possibile.
- Hai bisogno di logiche di routing (es. "se l'ordine è di tipo A vai al server 1, se è di tipo B vai al server 2").
- Vuoi una configurazione più semplice e immediata da gestire in Spring Boot. Reddit +5

Scegli Kafka se:

- Prevedi un volume di transazioni **altissimo** (migliaia al secondo).
- Vuoi fare **Event Sourcing**: ovvero tenere traccia di tutto lo storico dei movimenti per poter "ricostruire" lo stato del sistema in qualsiasi momento.
- Hai bisogno di analizzare i flussi di dati in tempo reale (es. per rilevamento frodi o analisi di mercato) mentre le transazioni avvengono. DEV Community +4

Il mio consiglio

Per un sistema di compravendita titoli dove la **correttezza del singolo ordine** è vitale e il volume non è quello di una borsa mondiale, [RabbitMQ](#) è solitamente la scelta più equilibrata e facile da implementare con Spring Boot. Se invece stai costruendo una piattaforma che deve gestire stream di dati massivi e storicizzazione infinita, [Apache Kafka](#) è lo standard industriale. Reddit +3

Su quale dei due vorresti vedere un **esempio di codice** per integrare il tuo server Spring Boot?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



Ok per RabbitMQ

22:52

Ottima scelta. **RabbitMQ** è perfetto per questo scenario perché è estremamente affidabile e gestisce nativamente i "acknowledgments" (conferme di ricezione), garantendo che nessun ordine di acquisto/vendita vada perso se il processore si blocca.

Ecco come configurare i due componenti separati.

1. Prerequisito: Dipendenza Maven

In entrambi i progetti (Server REST e Processor) aggiungi:

```
xml
```

AI Mode Tutti Immagini Video Notizie Altro ▾



2. Configurazione Comune (Nome Coda)



Assicurati che entrambi conoscano il nome della coda nel file

DEV Community

application.properties :

Qual è la differenza tra Kafka e RabbitMQ? - AWS

properties

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
trade.queue.name=trades-queue
```

messaggistica, inclusi anche molti esempi e note per l'uso

Usa il codice con cautela.



Nexsott

Mostra tutto

3. Server REST (Il Produttore)

Il server riceve la richiesta via REST, la salva su Postgres e "spara" un messaggio nella coda.

java

```
@Service
public class TradeProducer {

    @Autowired private RabbitTemplate rabbitTemplate;
    @Value("${trade.queue.name}") private String queueName;

    public void sendTrade(Long tradeId) {
        // Invia l'ID della transazione alla coda RabbitMQ
        rabbitTemplate.convertAndSend(queueName, tradeId);
    }
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



E

questo è un programma separato che ascolta la coda. Se il programma è spento, i messaggi rimangono in RabbitMQ e vengono elaborati appena lo riaccendi.

java

```
@Component
public class TradeConsumer {

    @Autowired private TradeRepository repository;

    @RabbitListener(queues = "${trade.queue.name}")
    @Transactional
    public void receiveTrade(Long tradeId) {
        System.out.println("Ricevuto ordine ID: " + tradeId);

        // 1. Carica la transazione da Postgres (DB Condiviso)
        TradeTransaction trade = repository.findById(tradeId).orElseThrow();

        try {
            // 2. Logica di acquisto/vendita titoli (chiamata broker esterno)
            executeMarketTrade(trade);

            // 3. Successo
            trade.setStatus(TransactionStatus.COMPLETED);
        } catch (Exception e) {
            // 4. Gestione errore
            trade.setStatus(TransactionStatus.FAILED);
        }

        repository.save(trade);
    }
}
```

AI Mode Tutti Immagini Video Notizie Altro ▾



configurato per rimettere il messaggio in coda e riprovare dopo X secondi.

3. **Parallelismo:** Se hai troppi ordini, puoi avviare **due o più istanze** del `Trade Processor`. RabbitMQ distribuirà gli ordini automaticamente (Round-Robin) tra le istanze disponibili.

Cosa ti serve per testarlo?

Devi avere RabbitMQ attivo. Il modo più veloce è usare **Docker**:

bash

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-man
```

Usa il codice con cautela.



(La porta 15672 ti permette di vedere una dashboard web bellissima con tutti i grafici delle transazioni in transito).

Ti interessa vedere come configurare il **meccanismo di Retry automatico** (ad esempio: se il broker titoli è offline, riprova 3 volte ogni 5 secondi)?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



... e Docker in questo contesto.

Immagina Docker come un "lettore DVD" e RabbitMQ come un "DVD". Invece di installare tutto il software, scarichi un'immagine pronta all'uso (il DVD) e la fai girare in un contenitore isolato (il lettore).

Come procedere in pratica:

1. **Installa Docker Desktop:** Vai sul sito ufficiale di [Docker](#) e installalo. È un'applicazione che rimane attiva nella barra delle icone.
2. **Lancia il comando:** Apri il terminale (o il Prompt dei comandi) e scrivi:

```
bash
```

```
docker run -d --name mio-rabbit -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

Usa il codice con cautela.



- `-d` : Fa girare RabbitMQ "in silenzio" (in background).
- `-p 5672:5672` : È la porta che serve a **Spring Boot** per inviare i messaggi.
- `-p 15672:15672` : È la porta che serve a **te** per vedere la pagina di controllo.
- `3-management` : È la versione che include l'interfaccia grafica.

3. **Controlla se funziona:** Apri il tuo browser e vai all'indirizzo

`http://localhost:15672` .

- User: `guest`
- Password: `guest`

AI Mode Tutti Immagini Video Notizie Altro ▾



modo che si colleghi correttamente a questo contenitore Docker?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ma su docker deve stare anche postgresql?

22:57

Non è obbligatorio, ma è fortemente consigliato in fase di sviluppo.

Ecco il motivo: se tieni sia **PostgreSQL** che **RabbitMQ** su Docker, il tuo ambiente di sviluppo diventa "portatile". Se domani cambi computer o lavori in team, ti basta un comando per avere tutto pronto, senza installare nulla sul sistema operativo.

Come gestire entrambi con un unico file

Invece di lanciare comandi singoli, la soluzione professionale è usare un file chiamato `docker-compose.yml`. Lo crei nella cartella principale del tuo progetto e ci scrivi dentro questo:

yaml

```
services:
```

AI Mode Tutti Immagini Video Notizie Altro ▾



```
image: rabbitmq:3-management
container_name: rabbitmq-broker
ports:
  - "5672:5672"    # Porta per Spring Boot
  - "15672:15672"  # Dashboard web
environment:
  RABBITMQ_DEFAULT_USER: guest
  RABBITMQ_DEFAULT_PASS: guest
```

Usa il codice con cautela.



Come si usa?

1. Apri il terminale nella cartella del file.
2. Scrivi: `docker-compose up -d`.
3. Magia: Docker accende contemporaneamente sia il database che il message broker.

E i tuoi programmi Spring Boot?

I tuoi due programmi Java (Server REST e Processor) gireranno **fuori** da Docker (sul tuo IDE come IntelliJ o Eclipse) e si collegheranno ai servizi così:

- **Postgres:** `localhost:5432`
- **RabbitMQ:** `localhost:5672`

Un piccolo avvertimento: Se hai già installato PostgreSQL direttamente su Windows/Mac, la porta `5432` sarà già occupata. In quel caso, o spegni il Postgres "nativo" o cambi la porta nel file yaml.

AI Mode Tutti Immagini Video Notizie Altro ▾



ma dove prendo il postgresql da mettere su docker?

22:59

Non devi scaricare alcun file manualmente! Questa è la comodità di Docker: ci pensa lui a scaricare tutto dai repository ufficiali (**Docker Hub**).

Quando scrivi `image: postgres:15` nel file `docker-compose.yml` (o quando lanci un comando `docker run`), Docker esegue questi passaggi:

1. Controlla se hai già l'immagine di PostgreSQL sul tuo PC.
2. Se non la trova, si collega automaticamente a Internet e la scarica dal **Docker Hub** (il "negoziario" ufficiale delle immagini, gratuito).
3. La avvia istantaneamente.

Ecco i passaggi pratici per avere tutto pronto:

1. **Crea il file:** Nella cartella del tuo progetto Java, crea un file di testo e nominalo esattamente `docker-compose.yml`.
2. **Incolla il codice:** Usa il codice che ti ho scritto nel messaggio precedente.
3. **Avvia:** Apri il terminale in quella cartella e digita:

```
bash
```

```
docker-compose up -d
```

Usa il codice con cautela.



AI Mode Tutti Immagini Video Notizie Altro ▾



```
# Configurazione PostgreSQL (Docker)
spring.datasource.url=jdbc:postgresql://localhost:5432/tradedb
spring.datasource.username=user
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update

# Configurazione RabbitMQ (Docker)
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

Usa il codice con cautela.



Un piccolo trucco

Se vuoi vedere se i dati stanno effettivamente entrando nel database PostgreSQL dentro Docker, puoi usare strumenti come **DBeaver** o **pgAdmin** collegandoti a `localhost:5432` con le credenziali che abbiamo scelto (`user` / `password`).

Hai già installato **Docker Desktop** sul tuo computer o preferisci una mano con l'installazione?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



Ecco i passaggi rapidi per essere operativo in 5 minuti:

1. Installazione

1. Vai sul sito ufficiale docker.com e scarica la versione per il tuo sistema operativo (Windows, Mac o Linux).
2. **Su Windows:** Durante l'installazione, assicurati che sia selezionata l'opzione **"Use WSL 2 instead of Hyper-V"** (è molto più veloce).
3. Al termine, riavvia il computer se richiesto.

2. Verifica se è attivo

Una volta riavviato, apri l'applicazione "Docker Desktop". Quando l'icona della balena (in basso a destra vicino all'orologio o nella barra delle applicazioni) diventa verde e smette di lampeggiare, Docker è pronto.

3. Primo test (importante)

Apri un terminale (PowerShell o CMD) e scrivi:

```
bash
```

```
docker --version
```

Usa il codice con cautela.



Se ti risponde con una versione (es. `Docker version 24.0.x`), sei pronto a procedere.

... ..

AI Mode Tutti Immagini Video Notizie Altro ▾



```
docker-compose up -d
```

Usa il codice con cautela.



In questo momento vedrai delle barre di caricamento: Docker sta scaricando PostgreSQL e RabbitMQ da internet per te.

5. Controlla i tuoi servizi

Apri l'interfaccia di Docker Desktop. Vedrai una riga con il nome della tua cartella: cliccandoci sopra, vedrai i due "contenitori" (`postgres-db` e `rabbitmq-broker`) con un pallino verde **Running**.

Vuoi che ti guidi nella creazione dei due progetti Spring Boot separati (Server e Processor) ora che l'infrastruttura è pronta?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



ma posso usare pgAdmin anche per collegarmi con il server su docker?

23:12

Certamente! pgAdmin vede il PostgreSQL che gira su Docker esattamente come se fosse installato direttamente sul tuo PC.

AI Mode Tutti Immagini Video Notizie Altro ▾



E

3. Tab "Connection" (Il punto cruciale)

Qui devi inserire i dati che abbiamo definito nel file `docker-compose.yml` :

- **Host name/address:** `localhost` (perché Docker "esporta" la porta sul tuo computer locale).
- **Port:** `5432`
- **Maintenance database:** `tradedb` (il nome che abbiamo dato nel file).
- **Username:** `user`
- **Password:** `password`

Clicca su **Save**.

Perché funziona?

Nel file `docker-compose.yml`, sotto la voce `ports`, abbiamo scritto `- "5432:5432"`. Questo comando dice a Docker: *"Prendi la porta 5432 interna al contenitore e rendila disponibile sulla porta 5432 del mio PC"*.

Quindi, per pgAdmin (e per i tuoi programmi Java), quel database si trova a tutti gli effetti su `localhost`.

Un consiglio utile

Se per caso hai già un altro PostgreSQL installato sul PC (non su Docker), pgAdmin potrebbe andare in conflitto. In quel caso:

1. Chiudi il Postgres locale.
2. Oppure, nel file `docker-compose.yml`, cambia la porta esterna in `"5433:5432"`.
A quel punto su pgAdmin userai la porta `5433`.

AI Mode Tutti Immagini Video Notizie Altro ▾



Ok

23:13

Ottimo! Ora hai le fondamenta pronte. Per rendere il tuo lavoro ancora più fluido, ecco come completare il file `docker-compose.yml` includendo anche **pgAdmin** direttamente nel "pacchetto". In questo modo, avviando Docker, avrai tutto il pannello di controllo pronto nel browser senza installare altro.

Il tuo file `docker-compose.yml` definitivo:

yaml

```
services:
  postgres:
    image: postgres:15
    container_name: postgres-db
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: tradedb

  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq-broker
    ports:
      - "5672:5672" # Porta per Spring Boot
      - "15672:15672" # Dashboard RabbitMQ (guest/guest)
```

AI Mode Tutti Immagini Video Notizie Altro ▾



PGADMIN_DEFAULT_PASSWORD: admin

Usa il codice con cautela.



Prossimi passi operativi:

1. **Avvia tutto:** Apri il terminale nella cartella del file e lancia `docker-compose up -d`.
2. **Verifica i pannelli di controllo:**
 - **RabbitMQ:** Vai su `http://localhost:15672` (User: `guest`, Pass: `guest`). Qui vedrai i messaggi passare in tempo reale.
 - **pgAdmin:** Vai su `http://localhost:8080` (Email: `admin@admin.com`, Pass: `admin`).
3. **Configura pgAdmin:** Una volta entrato in pgAdmin, aggiungi il server Postgres usando come host `postgres` (se sei dentro la rete Docker) oppure `host.docker.internal` (o semplicemente `localhost`).

Nota per Spring Boot

Ora che hai l'infrastruttura, il tuo lavoro si sposta sul codice Java. Dovrai creare:

1. Il **Server REST** che salva su DB e invia a RabbitMQ.
2. Il **Processor** che legge da RabbitMQ e aggiorna il DB.

Entrambi useranno le stesse credenziali che abbiamo scritto nel file sopra.

...

AI Mode Tutti Immagini Video Notizie Altro ▾



E

ok

23:15

Perfetto! Per creare i due progetti separati, il modo più veloce è usare **Spring Initializr** (start.spring.io).

Ecco come configurarli per farli comunicare tra loro:

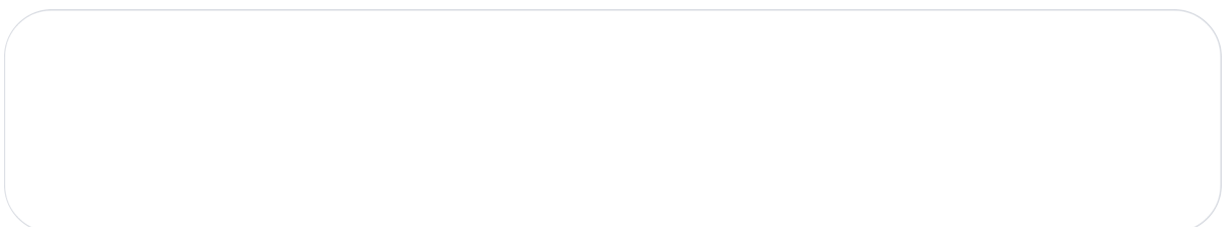
1. Creazione dei progetti

Dovrai generare due file `.zip` diversi con queste impostazioni:

Progetto A: `rest-server` (Il ricevitore)

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 3.x.x
- **Dependencies:**
 - `Spring Web` (per le API REST)
 - `Spring Data JPA` (per salvare su Postgres)
 - `PostgreSQL Driver`
 - `Spring RabbitMQ` (per inviare ordini)
 - `Lombok` (opzionale, per accorciare il codice)

Progetto B: `trade-processor` (L'elaboratore)



AI Mode Tutti Immagini Video Notizie Altro ▾



- (Nota: qui NON serve "Spring Web" se il processore non deve esporre API)

2. Configurazione della Coda (Bean Comune)

In **entrambi** i progetti, devi definire la coda. Crea una classe di configurazione:

java

```
@Configuration
public class RabbitConfig {
    @Bean
    public Queue tradeQueue() {
        // "true" rende la coda persistente: se Rabbit si spegne, non per
        return new Queue("trades-queue", true);
    }
}
```

Usa il codice con cautela.



3. Logica del REST Server (Invio)

Nel tuo controller o servizio nel `rest-server`:

java

```
@Autowired
private RabbitTemplate rabbitTemplate;
```

AI Mode Tutti Immagini Video Notizie Altro ▾



Nel `trade-processor`, crei un componente che "ascolta":

java

```
@Component
public class TradeListener {

    @RabbitListener(queues = "trades-queue")
    public void handleTrade(Long transactionId) {
        // Qui carichi dal DB PostgreSQL e fai l'operazione finanziaria
        System.out.println("Elaborazione transazione ID: " + transactionId);
    }
}
```

Usa il codice con cautela.



Un dettaglio importante: Il Database condiviso

Entrambi i progetti dovranno avere nel file `application.properties` la **stessa stringa di connessione** a PostgreSQL (quella che abbiamo impostato su Docker), così che uno possa scrivere la transazione "PENDING" e l'altro possa aggiornarla a "COMPLETED".

Vuoi che ti scriva un esempio completo della classe Entity e del Repository che dovrai condividere tra i due progetti?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)



AI Mode Tutti Immagini Video Notizie Altro ▾



E

Certamente, è un'operazione molto comune per testare il codice con dati reali. Esistono due modi principali per farlo:

1. Il metodo "Automatico" (all'avvio)

Se vuoi che il database venga popolato non appena lanci il comando `docker-compose up`, Docker ha una funzione utilissima: qualsiasi file `.sql` o `.sql.gz` messo nella cartella speciale `/docker-entrypoint-initdb.d/` all'interno del contenitore viene eseguito automaticamente.

Modifica così il tuo `docker-compose.yml`:

yaml

```
postgres:
  image: postgres:15
  container_name: postgres-db
  ports:
    - "5432:5432"
  volumes:
    # Copia il tuo dump dalla cartella locale 'backup' al contenitore
    - ./backup/mio_dump_prod.sql:/docker-entrypoint-initdb.d/init.sql
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: tradedb
```

Usa il codice con cautela.



2. Il metodo "Manuale" (a contenitore già acceso)

AI Mode Tutti Immagini Video Notizie Altro ▾



2. Esegui il ripristino (Restore):

bash

```
docker exec -it postgres-db psql -U user -d tradedb -f /tmp/backup.sql
```



Usa il codice con cautela.



Alcuni consigli importanti:

- **Versioni:** Assicurati che la versione di Postgres su Docker (es. 15) sia uguale o superiore a quella del database di produzione, altrimenti il dump potrebbe dare errori di compatibilità.
- **Dati Sensibili:** Trattandosi di transazioni finanziarie, se importi dati reali sul tuo PC, assicurati di anonimizzare (mascherare) i dati sensibili degli utenti (nomi, email, conti) per motivi di sicurezza e GDPR.
- **Dimensione:** Se il DB produttivo è enorme (molti GB), l'importazione potrebbe richiedere tempo. In quel caso, conviene fare un dump limitato alle ultime migliaia di righe.

Hai già un **file .sql** del database o devi ancora generarlo dal server di produzione?

Le risposte dell'AI potrebbero contenere errori. [Scopri di più](#)

AI Mode

Tutti

Immagini

Video

Notizie

Altro ▾

